Tutorial Prático: Implementando um Perceptron do Zero em Python

Disciplina: Inteligência Artificial
Aula 02: Do Perceptron ao Aprendizado de Máquina
Professor: Alexandre "Montanha" de Oliveira
Data: 26/02/2026

📋 Introdução

O que é um Perceptron?

O **perceptron** é o modelo mais simples de neurônio artificial, criado por Frank Rosenblatt em 1958. Imagine-o como um "tomador de decisões binário": ele recebe várias informações de entrada (inputs), pondera a importância de cada uma, e decide entre duas opções (sim/não, 0/1, verdadeiro/falso).

Analogia do Mundo Real:
Pense em um porteiro de uma festa VIP. Ele observa várias características de cada pessoa:

- Está na lista? (peso 0.8)
- Está bem vestida? (peso 0.5)
- Tem convite? (peso 0.9)

Se a soma ponderada dessas características ultrapassar um limite, a pessoa entra. Caso contrário, fica do lado de fora. Esse é exatamente o princípio do perceptron!

O que vamos implementar?

Neste tutorial, você irá:

1. ☑ **Construir** uma classe Perceptron do zero (sem bibliotecas de ML)
2. ☑ **Treinar** o perceptron para aprender funções lógicas (AND, OR)
3. ☑ **Testar** a limitação do modelo (problema XOR)
4. ☑ **Visualizar** como o perceptron "desenha" fronteiras de decisão
5. ☑ **Experimentar** com diferentes taxas de aprendizado
6. ☑ **Conectar** com conceitos modernos de Deep Learning

Requisitos

Software:

- Python 3.7 ou superior
- Bibliotecas: `numpy`, `matplotlib`

Plataforma:

- Google Colab (recomendado)

- Jupyter Notebook
- Qualquer IDE Python

Conhecimento Prévio:

- Conceitos básicos de Python
- Noções de álgebra (vetores, multiplicação)

Como executar no Google Colab

1. Acesse colab.research.google.com
2. Clique em "Novo notebook"
3. Copie e cole os códigos deste tutorial
4. Execute célula por célula com Shift+Enter

🧠 Parte 1: Entendendo a Matemática do Perceptron

Antes de programar, precisamos entender **como o perceptron funciona matematicamente**.

1.1 Os Componentes Fundamentais

Um perceptron possui:

1. **Entradas (x):** Os dados que alimentam o neurônio
   Exemplo: `x = [1, 0]` (dois valores de entrada)

2. **Pesos (w):** A importância de cada entrada
   Exemplo: `w = [0.5, -0.3]` (quanto cada entrada influencia na decisão)

3. **Bias (b):** Um ajuste adicional, como um "empurrãozinho"
   Exemplo: `b = 0.2`

4. **Função de Ativação:** A regra que transforma números em decisão (0 ou 1)

1.2 O Processo de Decisão (Forward Pass)

```
Passo 1: Calcular a soma ponderada
z = (x₁ × w₁) + (x₂ × w₂) + ... + (xₙ × wₙ) + b

Passo 2: Aplicar a função de ativação
ŷ = f(z) onde:
    f(z) = 1 se z ≥ 0
    f(z) = 0 se z < 0
```

Exemplo Numérico:

Dados:

- Entradas: `x = [1, 0]`

- Pesos: `w = [0.5, -0.3]`
- Bias: `b = 0.2`

Cálculo:

```
z = (1 × 0.5) + (0 × -0.3) + 0.2
z = 0.5 + 0 + 0.2
z = 0.7

Como z ≥ 0, então ŷ = 1
```

1.3 O Aprendizado (Ajuste de Pesos)

O perceptron aprende através da **Regra de Atualização de Pesos**:

```
Para cada peso w:
w_novo = w_antigo + η × (y_real - y_previsto) × x

Onde:
- η (eta) = taxa de aprendizado (ex: 0.1)
- y_real = resposta correta (rótulo verdadeiro)
- y_previsto = resposta dada pelo perceptron
- x = valor da entrada correspondente ao peso
```

Por que isso funciona?

- Se acertou: `(y_real - y_previsto) = 0` → pesos não mudam
- Se errou: `(y_real - y_previsto) ≠ 0` → pesos são ajustados na direção correta

Exemplo de Ajuste:

Situação:

- Peso atual: `w = 0.5`
- Entrada: `x = 1`
- Taxa de aprendizado: `η = 0.1`
- Esperado: `y_real = 1`
- Previsto: `y_previsto = 0` (errou!)

Atualização:

```
w_novo = 0.5 + 0.1 × (1 - 0) × 1
w_novo = 0.5 + 0.1
w_novo = 0.6
```

O peso aumentou porque o perceptron subestimou a resposta!

💻 Parte 2: Implementando a Classe Perceptron

Agora vamos traduzir toda essa matemática em código Python.

2.1 Preparação: Importar Bibliotecas

```
# Importar bibliotecas necessárias
import numpy as np
import matplotlib.pyplot as plt

# Configurar visualizações
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

# Fixar seed para reprodutibilidade
np.random.seed(42)

print("✅ Bibliotecas carregadas com sucesso!")
```

O que cada biblioteca faz:

- `numpy`: Operações matemáticas com vetores e matrizes (mais rápido que Python puro)
- `matplotlib`: Criar gráficos e visualizações

2.2 Implementação da Classe Perceptron

```
class Perceptron:
    """
    Implementação do Perceptron de Rosenblatt (1958)

    Um neurônio artificial que aprende a classificar dados linearmente
separáveis.
    """

    def __init__(self, learning_rate=0.1, n_iterations=100):
        """
        Inicializa o perceptron

        Parâmetros:
        -----------
        learning_rate : float
            Taxa de aprendizado (η) - controla o tamanho dos ajustes nos pesos
            Valores típicos: 0.01 a 1.0

        n_iterations : int
            Número máximo de épocas (passagens completas pelos dados de treino)
        """
        self.learning_rate = learning_rate
```

```python
        self.n_iterations = n_iterations
        self.weights = None  # Pesos (w) - serão inicializados no treino
        self.bias = None     # Bias (b) - será inicializado no treino
        self.errors_history = []  # Histórico de erros por época (para análise)

    def step_function(self, z):
        """
        Função de ativação: Função Degrau (Step Function)

        Converte a soma ponderada em uma decisão binária.
        É a função de ativação mais simples possível.

        Parâmetro:
        ----------
        z : float ou array
            Soma ponderada (produto escalar entre entradas e pesos + bias)

        Retorna:
        --------
        int ou array : 1 se z >= 0, caso contrário 0

        Por que usar essa função?
        - O perceptron original de Rosenblatt usava essa função
        - Produz saídas binárias (0 ou 1)
        - Simples de entender e implementar
        """
        return np.where(z >= 0, 1, 0)

    def predict(self, X):
        """
        Faz previsões para novos dados

        Processo:
        1. Calcula o produto escalar entre entradas (X) e pesos (w)
        2. Adiciona o bias (b)
        3. Aplica a função de ativação

        Parâmetro:
        ----------
        X : array de shape (n_amostras, n_features)
            Dados de entrada para classificar

        Retorna:
        --------
        array : Previsões (0 ou 1) para cada amostra

        Fórmula:
        --------
        ŷ = f(X · w + b)
        onde f é a função degrau
        """
        # Produto escalar: soma ponderada das entradas
        linear_output = np.dot(X, self.weights) + self.bias
```

```python
        # Aplicar função de ativação
        predictions = self.step_function(linear_output)

        return predictions

    def fit(self, X, y):
        """
        Treina o perceptron usando os dados fornecidos

        Algoritmo de Treinamento:
        1. Inicializar pesos e bias com zeros
        2. Para cada época:
            a. Para cada exemplo (x, y):
                - Calcular previsão ŷ
                - Calcular erro: e = y - ŷ
                - Atualizar pesos: w = w + η × e × x
                - Atualizar bias: b = b + η × e
        3. Repetir até convergir ou atingir n_iterations

        Parâmetros:
        -----------
        X : array de shape (n_amostras, n_features)
            Dados de treinamento (entradas)

        y : array de shape (n_amostras,)
            Rótulos verdadeiros (saídas esperadas: 0 ou 1)

        Retorna:
        --------
        self : objeto Perceptron treinado
        """
        n_samples, n_features = X.shape

        # Passo 1: Inicializar pesos e bias com zeros
        # Poderíamos usar valores aleatórios, mas zeros funcionam bem aqui
        self.weights = np.zeros(n_features)
        self.bias = 0

        # Imprimir estado inicial
        print(f"🎯 Iniciando treinamento...")
        print(f"   Pesos iniciais: {self.weights}")
        print(f"   Bias inicial: {self.bias}")
        print(f"   Taxa de aprendizado: {self.learning_rate}")
        print(f"   Número de amostras: {n_samples}")
        print("-" * 50)

        # Passo 2: Loop de treinamento (épocas)
        for epoch in range(self.n_iterations):
            errors = 0  # Contador de erros nesta época

            # Para cada exemplo de treinamento
            for idx, x_i in enumerate(X):
```

```
                # Fazer previsão para este exemplo
                # Reshape necessário pois predict espera array 2D
                prediction = self.predict(x_i.reshape(1, -1))[0]

                # Calcular erro
                error = y[idx] - prediction

                # Atualizar pesos se houve erro
                if error != 0:
                    # Regra de atualização do perceptron
                    update = self.learning_rate * error
                    self.weights += update * x_i
                    self.bias += update
                    errors += 1

            # Armazenar número de erros desta época
            self.errors_history.append(errors)

            # Mostrar progresso a cada 10 épocas
            if (epoch + 1) % 10 == 0:
                print(f"Época {epoch + 1:3d} | Erros: {errors:2d} | "
                      f"Pesos: {self.weights} | Bias: {self.bias:.4f}")

            # Critério de parada: se não houver erros, convergiu
            if errors == 0:
                print(f"\n✅ Convergência alcançada na época {epoch + 1}!")
                print(f"   Pesos finais: {self.weights}")
                print(f"   Bias final: {self.bias:.4f}")
                break

        # Se saiu do loop sem convergir
        if errors > 0:
            print(f"\n⚠️ Treinamento finalizado sem convergência completa")
            print(f"   Ainda havia {errors} erros na última época")

        return self

# Teste rápido da classe
print("\n" + "="*50)
print("CLASSE PERCEPTRON IMPLEMENTADA COM SUCESSO!")
print("="*50)
```

Pontos-chave da implementação:

1. `__init__`: Configura os hiperparâmetros (taxa de aprendizado e número de épocas)
2. `step_function`: Implementa a função de ativação (degrau)
3. `predict`: Faz previsões usando os pesos aprendidos
4. `fit`: Treina o modelo ajustando pesos baseado nos erros

🔬 Parte 3: Testando com o Problema AND

A função lógica AND retorna 1 apenas quando **ambas** as entradas são 1.

3.1 Entendendo o Problema AND

Tabela Verdade:

```
x₁ | x₂ | Saída (AND)
---|----|---------
0  | 0  |    0
0  | 1  |    0
1  | 0  |    0
1  | 1  |    1
```

Visualização Geométrica:
Os pontos podem ser separados por uma linha reta → **linearmente separável** ☑

```
x₂
 1 |   ×        ●  ← (1,1): classe 1
   |
 0 |   ×    ×      ← (0,0), (0,1), (1,0): classe 0
   └────────────
     0        1  x₁

× = classe 0 (AND retorna 0)
● = classe 1 (AND retorna 1)
```

3.2 Implementação: Treinando para AND

```python
print("\n" + "="*60)
print("EXPERIMENTO 1: PORTA LÓGICA AND")
print("="*60)

# Criar dataset para AND
X_and = np.array([
    [0, 0],  # Entrada 1
    [0, 1],  # Entrada 2
    [1, 0],  # Entrada 3
    [1, 1]   # Entrada 4
])

y_and = np.array([0, 0, 0, 1])  # Saídas esperadas

print("\n📊 Dataset AND:")
print("Entradas (X):")
print(X_and)
print("\nSaídas esperadas (y):")
print(y_and)
```

```
print("\n" + "-"*60)

# Criar e treinar o perceptron
perceptron_and = Perceptron(learning_rate=0.1, n_iterations=100)
perceptron_and.fit(X_and, y_and)

print("\n" + "-"*60)
print("📊 RESULTADOS DO TREINAMENTO")
print("-"*60)

# Fazer previsões
predictions_and = perceptron_and.predict(X_and)

print("\n🎯 Comparação: Esperado vs Previsto")
print("-"*40)
for i in range(len(X_and)):
    print(f"Entrada: {X_and[i]} | Esperado: {y_and[i]} | "
          f"Previsto: {predictions_and[i]} | "
          f"{'✅ CORRETO' if y_and[i] == predictions_and[i] else '❌
ERRADO'}")

# Calcular acurácia
accuracy_and = np.mean(predictions_and == y_and) * 100
print(f"\n🎯 Acurácia: {accuracy_and:.2f}%")
```

3.3 Visualizando a Fronteira de Decisão

```
def plot_decision_boundary(X, y, perceptron, title):
    """
    Plota os pontos de dados e a fronteira de decisão do perceptron

    A fronteira de decisão é a linha que separa as duas classes.
    Matematicamente: w₁x₁ + w₂x₂ + b = 0
    """
    # Configurar o gráfico
    plt.figure(figsize=(10, 7))

    # Plotar pontos de dados
    # Classe 0 (círculos vermelhos)
    plt.scatter(X[y == 0, 0], X[y == 0, 1],
                color='red', marker='o', s=200,
                edgecolors='black', linewidths=2,
                label='Classe 0', alpha=0.7)

    # Classe 1 (estrelas verdes)
    plt.scatter(X[y == 1, 0], X[y == 1, 1],
                color='green', marker='*', s=400,
                edgecolors='black', linewidths=2,
                label='Classe 1', alpha=0.7)
```

```
    # Calcular e plotar fronteira de decisão
    # A fronteira é definida por: w₁x₁ + w₂x₂ + b = 0
    # Resolvendo para x₂: x₂ = -(w₁x₁ + b) / w₂

    # Criar pontos x₁ de -0.5 a 1.5 (margem ao redor dos dados)
    x1_boundary = np.linspace(-0.5, 1.5, 100)

    # Calcular x₂ correspondentes
    # Evitar divisão por zero
    if perceptron.weights[1] != 0:
        x2_boundary = -(perceptron.weights[0] * x1_boundary + perceptron.bias)
/ perceptron.weights[1]
        plt.plot(x1_boundary, x2_boundary, 'b-', linewidth=2,
                label='Fronteira de Decisão')

    # Configurações do gráfico
    plt.xlim(-0.5, 1.5)
    plt.ylim(-0.5, 1.5)
    plt.xlabel('x₁ (Entrada 1)', fontsize=14, fontweight='bold')
    plt.ylabel('x₂ (Entrada 2)', fontsize=14, fontweight='bold')
    plt.title(title, fontsize=16, fontweight='bold', pad=20)
    plt.legend(loc='upper left', fontsize=12)
    plt.grid(True, alpha=0.3)
    plt.axhline(y=0, color='k', linewidth=0.5)
    plt.axvline(x=0, color='k', linewidth=0.5)

    # Adicionar anotações com os pesos
    equation = f'Fronteira: {perceptron.weights[0]:.2f}x₁ +
{perceptron.weights[1]:.2f}x₂ + {perceptron.bias:.2f} = 0'
    plt.text(0.02, 0.98, equation, transform=plt.gca().transAxes,
             fontsize=11, verticalalignment='top',
             bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

    plt.tight_layout()
    plt.show()

# Plotar resultado para AND
plot_decision_boundary(X_and, y_and, perceptron_and,
                       'Perceptron - Porta Lógica AND')

print("\n📊 Interpretação do Gráfico:")
print("-" * 60)
print("• A linha azul é a FRONTEIRA DE DECISÃO")
print("• Tudo acima/à direita da linha: perceptron prevê 1")
print("• Tudo abaixo/à esquerda da linha: perceptron prevê 0")
print("• Círculos vermelhos: classe 0 (AND retorna 0)")
print("• Estrelas verdes: classe 1 (AND retorna 1)")
print("\n✅ Note que a linha separa perfeitamente as duas classes!")
```

3.4 Analisando a Curva de Aprendizado

```
# Plotar evolução dos erros
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(perceptron_and.errors_history) + 1),
         perceptron_and.errors_history,
         marker='o', linewidth=2, markersize=8, color='blue')
plt.xlabel('Época', fontsize=14, fontweight='bold')
plt.ylabel('Número de Erros', fontsize=14, fontweight='bold')
plt.title('Curva de Aprendizado - AND', fontsize=16, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\n📊 Análise da Curva de Aprendizado:")
print("-" * 60)
print(f"• Número total de épocas: {len(perceptron_and.errors_history)}")
print(f"• Erros na época 1: {perceptron_and.errors_history[0]}")
print(f"• Erros na última época: {perceptron_and.errors_history[-1]}")
print(f"• Convergência: {'SIM ✅' if perceptron_and.errors_history[-1] == 0
else 'NÃO ❌'}")
print("\n💡 O que isso significa:")
print("  O perceptron começou errando algumas previsões,")
print("  mas aprendeu rapidamente e chegou a zero erros.")
print("  Isso acontece porque AND é linearmente separável!")
```

🔬 Parte 4: Testando com o Problema OR

A função lógica OR retorna 1 quando **pelo menos uma** das entradas é 1.

4.1 Entendendo o Problema OR

Tabela Verdade:

```
x₁ | x₂ | Saída (OR)
---|----|---------
0  | 0  |    0
0  | 1  |    1
1  | 0  |    1
1  | 1  |    1
```

Visualização Geométrica:
Também é **linearmente separável** ✅

```
x₂
 1 |   •       •  ← (0,1), (1,1): classe 1
   |
 0 |   ×   •      ← (0,0): classe 0; (1,0): classe 1
```

```
    └──────────
     0        1  x₁
```

4.2 Implementação: Treinando para OR

```python
print("\n" + "="*60)
print("EXPERIMENTO 2: PORTA LÓGICA OR")
print("="*60)

# Criar dataset para OR
X_or = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y_or = np.array([0, 1, 1, 1])  # Saídas esperadas

print("\n📊 Dataset OR:")
print("Entradas (X):")
print(X_or)
print("\nSaídas esperadas (y):")
print(y_or)
print("\n" + "-"*60)

# Criar e treinar o perceptron
perceptron_or = Perceptron(learning_rate=0.1, n_iterations=100)
perceptron_or.fit(X_or, y_or)

print("\n" + "-"*60)
print("📈 RESULTADOS DO TREINAMENTO")
print("-"*60)

# Fazer previsões
predictions_or = perceptron_or.predict(X_or)

print("\n🎯 Comparação: Esperado vs Previsto")
print("-"*40)
for i in range(len(X_or)):
    print(f"Entrada: {X_or[i]} | Esperado: {y_or[i]} | "
          f"Previsto: {predictions_or[i]} | "
          f"{'✅ CORRETO' if y_or[i] == predictions_or[i] else '❌ ERRADO'}")

# Calcular acurácia
accuracy_or = np.mean(predictions_or == y_or) * 100
print(f"\n🎯 Acurácia: {accuracy_or:.2f}%")

# Visualizar
```

```
plot_decision_boundary(X_or, y_or, perceptron_or,
                       'Perceptron - Porta Lógica OR')
```

4.3 Comparação: AND vs OR

```
print("\n" + "="*60)
print("COMPARAÇÃO: AND vs OR")
print("="*60)

print("\n📊 Pesos Finais:")
print("-" * 40)
print(f"AND - Peso 1: {perceptron_and.weights[0]:.4f}")
print(f"AND - Peso 2: {perceptron_and.weights[1]:.4f}")
print(f"AND - Bias:   {perceptron_and.bias:.4f}")
print()
print(f"OR  - Peso 1: {perceptron_or.weights[0]:.4f}")
print(f"OR  - Peso 2: {perceptron_or.weights[1]:.4f}")
print(f"OR  - Bias:   {perceptron_or.bias:.4f}")

print("\n💡 Interpretação:")
print("-" * 60)
print("• Para AND: pesos maiores e bias mais negativo")
print("  → Precisa de MAIS evidência para ativar (ambas entradas)")
print()
print("• Para OR: pesos menores e bias menos negativo")
print("  → Precisa de MENOS evidência para ativar (qualquer entrada)")
print()
print("• Ambos convergem rapidamente porque são linearmente separáveis!")
```

⚠ Parte 5: A Limitação do Perceptron - Problema XOR

Agora vamos testar o perceptron com um problema **impossível** para ele resolver.

5.1 Entendendo o Problema XOR

XOR (ou exclusivo) retorna 1 quando as entradas são **diferentes**.

Tabela Verdade:

```
x₁ | x₂ | Saída (XOR)
---|----|---------
0  | 0  |    0
0  | 1  |    1
1  | 0  |    1
1  | 1  |    0
```

Visualização Geométrica - O PROBLEMA:

```
x₂
 1 │   ×        ●  ← (0,1): classe 1; (1,1): classe 0
   │
 0 │   ×    ●      ← (0,0): classe 0; (1,0): classe 1
   └────────────
     0        1  x₁
```

❌ **IMPOSSÍVEL desenhar UMA ÚNICA linha reta que separe × de ●**

Este é um problema **NÃO linearmente separável**.

5.2 Implementação: Tentando Treinar para XOR

```
print("\n" + "="*60)
print("EXPERIMENTO 3: PORTA LÓGICA XOR")
print("="*60)

# Criar dataset para XOR
X_xor = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y_xor = np.array([0, 1, 1, 0])  # Saídas esperadas

print("\n📊 Dataset XOR:")
print("Entradas (X):")
print(X_xor)
print("\nSaídas esperadas (y):")
print(y_xor)
print("\n⚠️ ATENÇÃO: XOR é NÃO linearmente separável!")
print("   O perceptron NÃO deve conseguir aprender este padrão.")
print("\n" + "-"*60)

# Criar e treinar o perceptron
perceptron_xor = Perceptron(learning_rate=0.1, n_iterations=100)
perceptron_xor.fit(X_xor, y_xor)

print("\n" + "-"*60)
print("📈 RESULTADOS DO TREINAMENTO")
print("-"*60)

# Fazer previsões
predictions_xor = perceptron_xor.predict(X_xor)
```

```
print("\n🎯 Comparação: Esperado vs Previsto")
print("-"*40)
for i in range(len(X_xor)):
    print(f"Entrada: {X_xor[i]} | Esperado: {y_xor[i]} | "
          f"Previsto: {predictions_xor[i]} | "
          f"{'✅ CORRETO' if y_xor[i] == predictions_xor[i] else '❌
ERRADO'}")

# Calcular acurácia
accuracy_xor = np.mean(predictions_xor == y_xor) * 100
print(f"\n🎯 Acurácia: {accuracy_xor:.2f}%")

if accuracy_xor < 100:
    print("\n⚠️ FALHA ESPERADA!")
    print("   O perceptron não conseguiu aprender XOR perfeitamente.")

# Visualizar
plot_decision_boundary(X_xor, y_xor, perceptron_xor,
                       'Perceptron - Porta Lógica XOR (FALHA ESPERADA)')
```

5.3 Por que o Perceptron Falha no XOR?

```
print("\n" + "="*60)
print("POR QUE O PERCEPTRON FALHA NO XOR?")
print("="*60)

print("\n🔍 Análise Geométrica:")
print("-" * 60)
print("1. O perceptron só consegue desenhar LINHAS RETAS")
print("2. XOR precisa de uma separação em FORMATO DE CRUZ ou CURVA")
print("3. Matematicamente: XOR é NÃO linearmente separável")
print()
print("Visualizando:")
print()
print("  x₂")
print("   1│  [0]      [1]")
print("    │")
print("   0│  [0]      [1]")
print("    └──────────────  x₁")
print("       0        1")
print()
print("Impossível separar com uma linha reta!")
print()
print("💡 Solução Histórica:")
print("   • 1969: Minsky & Papert publicam 'Perceptrons'")
print("   • Provam matematicamente a limitação")
print("   • Causa o primeiro 'inverno da IA'")
print("   • Solução: Redes Neurais Multicamadas (1986)")

# Plotar curva de aprendizado do XOR
```

```
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(perceptron_xor.errors_history) + 1),
         perceptron_xor.errors_history,
         marker='o', linewidth=2, markersize=8, color='red')
plt.xlabel('Época', fontsize=14, fontweight='bold')
plt.ylabel('Número de Erros', fontsize=14, fontweight='bold')
plt.title('Curva de Aprendizado - XOR (Não Converge)',
         fontsize=16, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\n📊 Análise da Curva:")
print("-" * 60)
print("• Note que o número de erros NÃO chega a zero")
print("• O perceptron fica 'preso' tentando aprender")
print("• Não há configuração de pesos que resolva XOR com uma linha")
print("• Este foi um resultado CRUCIAL na história da IA")
```

🔬 Parte 6: Experimentando com Taxa de Aprendizado

A **taxa de aprendizado (η)** controla o tamanho dos passos durante o aprendizado.

6.1 Impacto da Taxa de Aprendizado

```
print("\n" + "="*60)
print("EXPERIMENTO 4: IMPACTO DA TAXA DE APRENDIZADO")
print("="*60)

# Testar diferentes taxas de aprendizado
learning_rates = [0.01, 0.1, 1.0]
colors = ['blue', 'green', 'red']

plt.figure(figsize=(14, 5))

for idx, lr in enumerate(learning_rates):
    print(f"\n{'='*60}")
    print(f"Taxa de Aprendizado: {lr}")
    print(f"{'='*60}")

    # Treinar perceptron com esta taxa
    perceptron_lr = Perceptron(learning_rate=lr, n_iterations=100)
    perceptron_lr.fit(X_and, y_and)

    # Plotar curva de aprendizado
    plt.subplot(1, 3, idx + 1)
    plt.plot(range(1, len(perceptron_lr.errors_history) + 1),
             perceptron_lr.errors_history,
             marker='o', linewidth=2, markersize=6, color=colors[idx])
```

```
    plt.xlabel('Época', fontsize=12, fontweight='bold')
    plt.ylabel('Erros', fontsize=12, fontweight='bold')
    plt.title(f'η = {lr}', fontsize=14, fontweight='bold')
    plt.grid(True, alpha=0.3)
    plt.ylim(-0.5, 5)

plt.tight_layout()
plt.show()

print("\n" + "="*60)
print("ANÁLISE DOS RESULTADOS")
print("="*60)

print("\n🐢 Taxa Baixa (η = 0.01):")
print("-" * 60)
print("• Convergência LENTA")
print("• Passos muito pequenos")
print("• Seguro, mas ineficiente")
print("• Use quando: dados ruidosos ou instabilidade")

print("\n⚖️ Taxa Moderada (η = 0.1):")
print("-" * 60)
print("• EQUILÍBRIO ideal")
print("• Convergência rápida e estável")
print("• Valor PADRÃO recomendado")
print("• Use quando: situação típica")

print("\n🚀 Taxa Alta (η = 1.0):")
print("-" * 60)
print("• Convergência muito rápida OU instabilidade")
print("• Passos grandes podem 'pular' a solução")
print("• Pode oscilar sem convergir em problemas complexos")
print("• Use quando: problema simples e quer velocidade")

print("\n💡 Regra Prática:")
print("-" * 60)
print("• Comece com η = 0.1 (valor padrão)")
print("• Se não convergir: reduza (ex: 0.01)")
print("• Se convergir muito devagar: aumente (ex: 0.5)")
print("• Problemas difíceis: taxas menores (0.001 - 0.1)")
print("• Problemas simples: taxas maiores (0.1 - 1.0)")
```

6.2 Análise Comparativa Detalhada

```
# Criar tabela comparativa
print("\n" + "="*60)
print("TABELA COMPARATIVA: IMPACTO DA TAXA DE APRENDIZADO")
print("="*60)

results = []
```

```
for lr in [0.001, 0.01, 0.05, 0.1, 0.5, 1.0]:
    perceptron_test = Perceptron(learning_rate=lr, n_iterations=100)
    perceptron_test.fit(X_and, y_and)

    results.append({
        'Taxa': lr,
        'Épocas': len(perceptron_test.errors_history),
        'Convergiu': perceptron_test.errors_history[-1] == 0,
        'Erros_Finais': perceptron_test.errors_history[-1]
    })

print("\n{:<10} {:<10} {:<12} {:<15}".format(
    "Taxa (η)", "Épocas", "Convergiu?", "Erros Finais"
))
print("-" * 60)

for r in results:
    print("{:<10} {:<10} {:<12} {:<15}".format(
        r['Taxa'],
        r['Épocas'],
        "✅ Sim" if r['Convergiu'] else "❌ Não",
        r['Erros_Finais']
    ))

print("\n" + "="*60)
```

🧠 Parte 7: Conceitos Avançados e Conexão com IA Moderna

7.1 Do Perceptron Simples às Redes Neurais Profundas

```
print("\n" + "="*60)
print("DO PERCEPTRON AO DEEP LEARNING")
print("="*60)

print("""
📜 LINHA DO TEMPO E EVOLUÇÃO:

1943 - McCulloch & Pitts
│      Primeiro modelo matemático de neurônio
│
1958 - Rosenblatt
│      PERCEPTRON (o que implementamos hoje!)
│      ✅ Aprende padrões lineares
│      ❌ Não resolve XOR
│
1969 - Minsky & Papert
│      Livro "Perceptrons" → Primeiro Inverno da IA
│      Provam limitações matemáticas
```

```
│
1986 - Rumelhart, Hinton & Williams
│      BACKPROPAGATION + Redes Multicamadas
│      ☑ Resolve XOR com camada oculta
│      ☑ Funções não-lineares
│
2012 - Krizhevsky (AlexNet)
│      Deep Learning vence ImageNet
│      Revolução da IA moderna
│
2017-2024 - Era Transformer
│      GPT, BERT, ChatGPT, Gemini
│      Bilhões de parâmetros
│      Mas o PRINCÍPIO é o mesmo! ⚡

""")

print("🧠 Como o Perceptron se Conecta com Redes Modernas:")
print("-" * 60)
print()
print("1. NEURÔNIO BÁSICO (Perceptron):")
print("   ŷ = f(w·x + b)")
print()
print("2. REDE NEURAL MULTICAMADAS:")
print("   • Empilhar múltiplos perceptrons")
print("   • Adicionar camadas ocultas")
print("   • Resolver problemas não-lineares (XOR!)")
print()
print("3. DEEP LEARNING:")
print("   • Muitas camadas (10, 50, 100+)")
print("   • Milhões/bilhões de pesos")
print("   • Mesma regra de atualização (gradient descent)")
print()
print("💡 CONCLUSÃO:")
print("   O perceptron é o 'átomo' das redes neurais modernas!")
print("   Tudo que você aprendeu hoje escala para GPT, BERT, etc.")
```

7.2 Como Resolver XOR com Redes Neurais

```
print("\n" + "="*60)
print("COMO RESOLVER XOR: VISÃO GERAL")
print("="*60)

print("""
❌ Perceptron Simples (1 camada):

   x₁ ──────┐
            ├──→ Σ ──→ f() ──→ ŷ
   x₂ ──────┘
```

```
   Pode:
   • AND, OR, NOT
   • Qualquer função linearmente separável

   Não Pode:
   • XOR
   • Problemas complexos

✅ Rede Neural Multicamadas (2+ camadas):

            Camada Oculta    Camada Saída
   x₁ ──────┬──→ h₁ ──────┐
            │              ├──→ ŷ
   x₂ ──────├──→ h₂ ──────┘
            │
            └──→ h₃

   Pode:
   • XOR ✓
   • Funções não-lineares complexas ✓
   • Classificação de imagens ✓
   • Reconhecimento de fala ✓

🔑 DIFERENÇA FUNDAMENTAL:

   • Camada oculta cria REPRESENTAÇÕES INTERMEDIÁRIAS
   • Neurônios ocultos aprendem FEATURES úteis
   • Combina múltiplas fronteiras lineares
   • Resultado: capacidade de modelar NÃO-LINEARIDADE

📚 Para Aprender Mais:

   • Próximas aulas: implementaremos MLP (Multi-Layer Perceptron)
   • Veremos backpropagation (como treinar redes profundas)
   • Aplicações práticas: MNIST, classificação de imagens
""")
```

7.3 Limitações e Quando NÃO Usar Perceptron

```
print("\n" + "="*60)
print("QUANDO USAR (E NÃO USAR) PERCEPTRON")
print("="*60)

print("\n✅ USE PERCEPTRON QUANDO:")
print("-" * 60)
print("• Problema é linearmente separável")
print("• Classificação binária simples")
print("• Poucos features (< 100)")
print("• Precisa de modelo interpretável")
print("• Dados pequenos/limitados")
```

```
print("• Quer entender fundamentos de ML")
print()
print("Exemplos:")
print("  → Classificar emails (spam: sim/não) com palavras-chave")
print("  → Aprovar crédito baseado em 3-4 variáveis simples")
print("  → Detectar fraude com regras lineares")

print("\n❌ NÃO USE PERCEPTRON QUANDO:")
print("-" * 60)
print("• Problema é não-linear (ex: XOR)")
print("• Múltiplas classes (>2)")
print("• Dados complexos (imagens, áudio, texto)")
print("• Padrões sutis e hierárquicos")
print("• Grande volume de dados")
print("• Estado da arte é necessário")
print()
print("Exemplos:")
print("  → Reconhecimento facial → Use CNN (Redes Convolucionais)")
print("  → Análise de sentimento de texto → Use Transformers")
print("  → Jogar xadrez → Use Redes Profundas + RL")
print("  → Classificar 1000 objetos → Use ResNet, EfficientNet")

print("\n🎯 REGRA DE OURO:")
print("-" * 60)
print("  Se você pode visualizar os dados em 2D/3D")
print("  e separar com uma linha/plano,")
print("  então o perceptron PODE funcionar.")
print()
print("  Se não consegue visualizar uma separação linear,")
print("  você PRECISA de redes mais profundas.")
```

🎯 Parte 8: Resumo e Checklist de Aprendizado

8.1 Conceitos-Chave Aprendidos

```
print("\n" + "="*60)
print("CHECKLIST: O QUE VOCÊ APRENDEU HOJE")
print("="*60)

checklist = [
    ("Conceito de neurônio artificial",
     "Unidade básica que processa informação"),

    ("Arquitetura do perceptron",
     "Entradas, pesos, bias, soma ponderada, ativação"),

    ("Função de ativação (step function)",
     "Converte números em decisões binárias"),
```

```
    ("Regra de aprendizado do perceptron",
     "w = w + η × erro × x"),

    ("Implementação do zero em Python",
     "Classe completa sem bibliotecas de ML"),

    ("Problemas linearmente separáveis",
     "AND, OR - perceptron RESOLVE"),

    ("Problemas não-linearmente separáveis",
     "XOR - perceptron FALHA"),

    ("Taxa de aprendizado (η)",
     "Controla velocidade e estabilidade"),

    ("Fronteira de decisão",
     "Linha que separa classes"),

    ("Convergência",
     "Quando o modelo para de errar"),

    ("Limitação histórica",
     "Inverno da IA (1969) por causa do XOR"),

    ("Conexão com Deep Learning",
     "Perceptron é o bloco básico de redes modernas")
]

for i, (concept, description) in enumerate(checklist, 1):
    print(f"\n{i:2d}. ☑ {concept}")
    print(f"    → {description}")

print("\n" + "="*60)
```

8.2 Exercícios Propostos

```
print("\n" + "="*60)
print("EXERCÍCIOS PARA PRATICAR")
print("="*60)

print("""
🎯 NÍVEL BÁSICO (Consolidar Conceitos):

1. Modifique a taxa de aprendizado para 0.5 e treine AND
   • Compare com η=0.1
   • Qual converge mais rápido?

2. Implemente a função lógica NAND (NOT AND)
   • Tabela verdade: (0,0)→1, (0,1)→1, (1,0)→1, (1,1)→0
   • O perceptron consegue aprender?
```

```
3. Teste o perceptron com dados NOR (NOT OR)
   • Tabela verdade: (0,0)→1, (0,1)→0, (1,0)→0, (1,1)→0
   • Plote a fronteira de decisão


🎯 NÍVEL INTERMEDIÁRIO (Explorar Limites):

4. Crie um dataset com 3 features (ao invés de 2)
   • Exemplo: [x1, x2, x3] → y
   • Treine o perceptron
   • Como visualizar em 3D?

5. Adicione ruído aos dados
   • Use: X + np.random.normal(0, 0.1, X.shape)
   • O perceptron ainda converge?
   • Teste diferentes níveis de ruído

6. Implemente uma função de ativação diferente
   • Sigmoid: σ(z) = 1 / (1 + e^(-z))
   • Compare com step function


🎯 NÍVEL AVANÇADO (Aprofundar):

7. Implemente validação cruzada
   • Divida dados em treino/teste
   • Avalie generalização

8. Adicione regularização L2
   • Penalizar pesos grandes
   • w = w - η × λ × w (a cada época)

9. Crie visualização interativa
   • Use ipywidgets no Colab
   • Sliders para η, número de épocas
   • Atualização em tempo real

10. Pesquise o ADALINE (Adaptive Linear Neuron)
    • Diferença para o Perceptron
    • Implemente e compare
""")

print("\n" + "="*60)
```

8.3 Recursos para Estudo Adicional

```
print("\n" + "="*60)
print("RECURSOS PARA APROFUNDAMENTO")
print("="*60)
```

```
print("""
📚 LEITURA RECOMENDADA:

• Livros:
  1. "Neural Networks and Deep Learning" - Michael Nielsen
     → http://neuralnetworksanddeeplearning.com (GRATUITO!)

  2. "Deep Learning" - Goodfellow, Bengio, Courville
     → https://www.deeplearningbook.org (GRATUITO!)

  3. "Make Your Own Neural Network" - Tariq Rashid
     → Excelente para iniciantes

• Papers Históricos:
  1. Rosenblatt (1958) - "The Perceptron"
  2. Minsky & Papert (1969) - "Perceptrons"
  3. Rumelhart et al. (1986) - "Learning representations by backpropagation"


🎥 VÍDEOS:

• 3Blue1Brown - Neural Networks Series
  → Visualizações INCRÍVEIS de como redes neurais funcionam

• StatQuest - Neural Networks Clearly Explained
  → Didático e acessível

• Andrew Ng - Machine Learning Course (Coursera)
  → Curso completo, gratuito para auditar


💻 CÓDIGO E PRÁTICA:

• Kaggle - Learn Machine Learning
  → Tutoriais interativos gratuitos

• Google Colab - Notebooks prontos
  → Busque "perceptron implementation"

• GitHub - Projetos de exemplo
  → Procure: "perceptron python implementation"


🔧 FERRAMENTAS PROFISSIONAIS:

• scikit-learn: biblioteca de ML do Python
  → from sklearn.linear_model import Perceptron

• TensorFlow/PyTorch: frameworks de Deep Learning
  → Próximos passos após dominar o básico

• Keras: API de alto nível para redes neurais
```

```
    → Facilita construir modelos complexos
  """)

  print("\n" + "="*60)
```

🎓 Conclusão

Parabéns! 🎉 Você acabou de implementar e compreender o **perceptron**, o bloco fundamental da Inteligência Artificial moderna.

O que você dominou:

- ☑ **Implementação do zero** - Sem "caixas-pretas", você entende cada linha
- ☑ **Matemática aplicada** - Vetores, produtos escalares, gradiente
- ☑ **Limitações reais** - XOR e a necessidade de redes mais profundas
- ☑ **Hiperparâmetros** - Taxa de aprendizado e convergência
- ☑ **Conexão histórica** - De 1958 ao ChatGPT

Próximos passos na disciplina:

Na **Aula 03**, vamos evoluir o perceptron para **Redes Neurais Multicamadas**, e você verá como resolver o problema XOR usando camadas ocultas!

📌 Código Completo para Download

Você pode copiar todo o código deste tutorial e executar em um único notebook Python. Todos os blocos de código estão prontos para execução sequencial.

Sugestão de organização:

1. Crie um novo notebook no Google Colab
2. Copie cada seção em células separadas
3. Execute célula por célula (Shift+Enter)
4. Experimente modificar valores e testar suas hipóteses!

💬 Contato e Dúvidas

Professor: Alexandre "Montanha" de Oliveira
Disciplina: Inteligência Artificial - 2026/1

Dúvidas? Use o fórum da disciplina ou envie mensagem pelo ambiente virtual!

✨ **"A jornada de mil camadas começa com um único perceptron."** ✨

Documento gerado para fins educacionais | UniBH 2026